

SOC ANALYST LEVEL 1

Lecture 10: KQL Basics for SOC Analyst Level 1

How a SOC Analyst writes basic queries to search and investigate inside a
SIEM

Author: [Nader Anam](#) | Duration: 3
Hours

LECTURE OBJECTIVES

- ✓ Understand the meaning of KQL.
- ✓ Know why a SOC analyst needs KQL.
- ✓ Understand basic query structure.
- ✓ Use take.
- ✓ Use where.
- ✓ Use contains.
- ✓ Use summarize.
- ✓ Use count.
- ✓ Use ago.
- ✓ Search for Failed Logins.
- ✓ Search for Successful Logins.
- ✓ Extract users with the most login failures.
- ✓ Build initial investigative queries.
- ✓ Read query results and translate them into security analysis.

WHY IS THIS LECTURE IMPORTANT?

In the previous lecture, we learned about SIEM. But having a SIEM alone is not enough.

A SOC analyst needs to ask the system questions like:

- ❓ Whose login failed?
- ❓ Who successfully logged in after a failure?
- ❓ What IP attempted to log in the most?
- ❓ Did this user appear in the last 24 hours?
- ❓ Did this device connect to a suspicious address?
- ❓ Is there encoded PowerShell executing?
- ❓ Has this Hash appeared on other devices?

KQL is the method we use to ask these questions inside environments like Microsoft Sentinel.

| WHAT IS KQL?

KQL stands for **Kusto Query Language**.

It is a query language used to search and analyze massive datasets like Logs and Events.

From a SOC perspective, KQL helps the analyst to:

Search inside logs & filter results

Count events & aggregate data

Build Timelines & discover patterns

Support active investigations

Write data-driven reports

| MUST AN L1 ANALYST BE A KQL EXPERT?

No.

A SOC Level 1 analyst does not need to be a KQL engineer. However, they need to know the practical basics:

-  How to display initial results.
-  How to filter by EventID.
-  How to search for a user.
-  How to search within the last 24 hours.
-  How to group results by User or IP.
-  How to read the output and write an analysis.

The goal is not writing complex queries. The goal is searching correctly.

IMPORTANT RULE BEFORE WRITING KQL

Do not start by writing a random query. Start with a clear security question.

✗ Weak Example:

"I want to search in SecurityEvent."

✓ Good Example:

"I want to know which users had the most failed logins in the last 24 hours."

A good question produces a good query.

BASIC KQL STRUCTURE

A KQL query usually starts with a table name, followed by the Pipe character | :

```
SecurityEvent  
| 10  
take
```

The Concept:

SecurityEvent is the table name.

| (Pipe) passes the results to the next step.

take 10 displays only 10 records.

The | sign is extremely important in KQL.

WHAT IS A TABLE?

A **Table** contains records of a specific type.

Examples of tables in SOC environments:

SecurityEvent

SignInLogs

DeviceProcessEvents

DeviceNetworkEvents

EmailEvents

CommonSecurityLog

Syslog

AzureActivity

Note: Table names vary depending on the environment and connected log sources.

| THE SECURITYEVENT TABLE

The SecurityEvent table is mostly used for Windows Security Events.

Examples of Events inside it:

4624 - Successful Logon

4625 - Failed Logon

4672 - Special Privileges Assigned

4688 - Process Creation

4720 - User Created

4728 - User Added to Security Group

1102 - Audit Log Cleared

In this lecture, we will use SecurityEvent frequently because it is ideal for beginners.

THE FIRST QUERY

```
SecurityEvent  
|  
10  
take
```

Meaning: Show 10 records from the SecurityEvent table.

Usage:

To verify the table has data.

To see what the fields look like.

To see quick examples.

As a starting point for exploration.

Note: This is not a real investigative query, but it is a good start to explore the table.

USING TAKE

take displays a specific number of records.

```
SecurityEvent
|
5
take
```

```
SecurityEvent
|
100
take
```

When do we use it?

When exploring a new table.

When testing a query.

To avoid displaying thousands of results.

But do not rely on take alone for investigation because it only shows a random sample.

USING WHERE

where is used for filtering.

```
SecurityEvent
  EventID == 4625
where
```

Meaning: Show only events where the EventID equals 4625.

In Windows: **4625 = Failed Logon**

This is your first practical query to analyze failed login attempts.

THE DIFFERENCE BETWEEN = AND ==

In KQL, we use `==` for comparison.

✓Correct Example:

```
SecurityEvent
|      EventID == 4625
where
```

✗Do Not Use:

```
      EventID = 4625
|
where
```

Because `=` is not the correct comparison operator in this context.

SEARCHING FOR SUCCESSFUL LOGON

```
SecurityEvent  
| EventID == 4624  
where
```

Meaning: Show successful login events. (4624 = Successful Logon)

Usage in SOC:

To check if a successful login occurred.

Searching after observing Failed Logins.

Inspecting the Logon Type.

Inspecting the Source IP.

Inspecting the User.

SEARCHING FOR FAILED LOGON

```
SecurityEvent  
|      EventID == 4625  
where
```

Meaning: Show failed login events. (4625 = Failed Logon)

Usage in SOC:

Detecting Brute Force attacks.

Detecting Password Spraying.

Finding login attempts on sensitive accounts.

Inspecting the attacking Source IP.

Checking the volume/number of attempts.

FILTERING BY TIME

Usually, we don't want to search through all logs. We might just want the last 24 hours:

```
SecurityEvent
|      TimeGenerated > ago(24h)
where
```

Meaning: Show events that occurred within the last 24 hours.

This is extremely important to reduce the dataset and improve search speed.

FAILED LOGINS IN THE LAST 24 HOURS

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where
where
```

Meaning: Show Failed Logins that happened within the last 24 hours.

This is a highly practical and useful query for a SOC analyst.

ORDERING THE FILTERS

You can write the query stacked like this:

```
SecurityEvent
  TimeGenerated > ago(24h)
|
EventID == 4625
where
where
```

Or combined using and like this:

```
SecurityEvent
  TimeGenerated > ago(24h) and EventID == 4625
|
where
```

Both are correct. For beginners, it is better to split conditions into multiple lines to keep the query clean and easy to read.

SEARCHING FOR A SPECIFIC USER

```
SecurityEvent
  EventID == 4625
  Account contains "ahmed"
where
where
```

Meaning: Show failed login attempts where the account name contains the word "ahmed".

contains is very useful when you do not know the exact full name of the account.

USING CONTAINS

contains searches within text strings. (It is case-insensitive).

```
SecurityEvent
|   Account contains "admin"
where
```

Meaning: Show events where the Account field contains the word "admin".

Usage in SOC:

Searching for Admin accounts.

Searching for partial usernames.

Searching inside a CommandLine.

Searching for a file name or a Domain.

SEARCHING FOR ADMIN ACCOUNTS

```
SecurityEvent
|   TimeGenerated > ago(24h)
where Account contains "admin"
where
```

Usage:

Checking the activity of administrative accounts.

Looking for Failed Logins on sensitive accounts.

Tracking the usage of highly privileged accounts.

Warning: This might return many irrelevant results. Always tie your search to a clear question.

FAILED LOGINS FOR ADMIN ACCOUNTS

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where  Account contains "admin"
where
```

Analysis:

This query searches for failed login attempts on any account containing "admin" over the last 24 hours.

This is critical because targeting Admin accounts is much more dangerous than targeting a standard user account.

USING SUMMARIZE

summarize is used to aggregate and group results.

```
SecurityEvent
|      EventID == 4625
|      Count=count() by Account
where
summarize
```

Meaning: Count the number of Failed Logins for each Account.

Without summarize, you will just see thousands of individual raw events. With summarize, you see a clean summary that aids analysis.

USING COUNT

count() calculates the total number of records.

```
SecurityEvent
  EventID == 4625
|
FailedCount=count()
where
summarize
```

Meaning: Calculate the total absolute number of failed login attempts.

This gives a general number, but it doesn't tell us *which* users are affected.

Therefore, we mostly use: **by Account**

USERS WITH THE MOST FAILED LOGINS

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where  FailedCount=count() by Account
where  FailedCount desc
|      sort
summarize
by
```

Meaning: Show the accounts with the highest number of failed logins in the last 24 hours.

This helps detect:

Brute Force attacks.

Targeted accounts.

A misconfigured service using a bad password.

Password Spraying.

USING SORT BY

sort by arranges the results.

```
| sort      FailedCount desc  
by
```

Meaning: Sort the results in descending order (highest to lowest).

In the SOC, we use it to find out:

- The most active attacking IP.

- The most affected account.

- The host generating the most alerts.

- The most frequently occurring EventID.

SOURCE IPS CAUSING THE MOST FAILED LOGINS

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where  FailedCount=count() by IPAddress
where  FailedCount desc
| sort
| summarize
by
```

Meaning: Show the IP addresses that caused the most failed login attempts.

This is incredibly useful for spotting an attacking IP or identifying a misconfigured internal source.

THE POTENTIAL ISSUE WITH IPADDRESS

Some environments might not use a field exactly named `IpAddress`.

The field could be named:

`IpAddress`

`SourceIP`

`SourceIpAddress`

`ClientIP`

`SourceNetworkAddress`

Therefore, your first step when investigating a new table should be:

```
SecurityEvent
```

```
10
```

SEARCHING FOR SPECIFIC EVENTID WITH USER & IP

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where  TimeGenerated, Account, Computer, IPAddress, Activity
where
project
```

Meaning: Show only the important columns.

Instead of viewing all 50+ background fields, you neatly display:

Time

Account

Computer

IP Address

Activity (The event description)

USING PROJECT

project is used to select specific columns you want to display.

```
| TimeGenerated, Account, Computer, EventID  
project
```

Why is this important?

- It reduces noise.

- It makes the results much easier to read.

- It helps when copying data into an incident report.

- It makes your analysis significantly faster.

SUCCESSFUL LOGIN AFTER FAILED LOGINS

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID in (4624, 4625)
where  Account contains "ahmed"
where  TimeGenerated, EventID, Account, Computer, IPAddress, Activity
where  TimeGenerated asc
project
by
```

Meaning: Show successful and failed login events for the user "ahmed" during the last 24 hours, ordered chronologically (oldest to newest).

This query helps you see if a successful login immediately followed a string of failures.

USING IN

in is used to search for multiple matching values at once.

```
|      EventID in (4624, 4625)  
where
```

Meaning: Show events where the EventID is either 4624 OR 4625.

This is highly useful when building a Timeline that needs to capture multiple types of related events simultaneously.

BUILDING A SIMPLE TIMELINE

```
SecurityEvent
|   TimeGenerated > ago(24h)
|   Account contains "finance.manager"
where EventID in (4624, 4625, 4672, 4688)
where TimeGenerated, EventID, Account, Computer, IPAddress, Process, CommandLine
where TimeGenerated asc
project
by
```

Goal: To see the exact chronological sequence:

Failed Login → Successful Login → Special Privileges → Process Creation

NOTE ON PROCESS AND COMMANDLINE FIELDS

Not every log in SecurityEvent will contain a Process or CommandLine field.

You might need to query a different table, such as:

- DeviceProcessEvents

- Sysmon logs

- Microsoft Defender logs

The Rule: If you cannot find a field in one table, do not assume the data is missing from the environment. It might just be stored in a different table.

SEARCHING FOR POWERSHELL

A general query example:

```
SecurityEvent
| TimeGenerated > ago(24h)
| EventID == 4688
where Process contains "powershell"
where
```

Meaning: Search for Process Creation events where the process involves PowerShell.

Note: In some environments, EventID 4688 might not be enabled, or the field name for the process might differ.

SEARCHING FOR ENCODED POWERSHELL

```
SecurityEvent
| TimeGenerated > ago(24h)
| EventID == 4688
where CommandLine contains "-enc" or CommandLine contains "EncodedCommand"
where
```

Meaning: Search for PowerShell executions containing encoded commands.

This is a strong indicator of compromise, but it does not prove an attack on its own. You must investigate:

Parent Process | User | Host | Network connections | Email activity

USING OR

or is used when we want ANY of the specified conditions to be true.

```
|      CommandLine contains "-enc" or CommandLine contains "EncodedCommand"  
where
```

Meaning: Display the results if the string contains "-enc" OR if it contains "EncodedCommand".

USING AND

and means that ALL specified conditions must be true.

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625 and Account contains "admin"
where
where
```

Meaning: Display Failed Logins ONLY for accounts whose name contains the word "admin". Both conditions must match.

SEARCHING FOR AUDIT LOG CLEARED

```
SecurityEvent
|      TimeGenerated > ago(7d)
|      EventID == 1102
where  TimeGenerated, Account, Computer, Activity
where  TimeGenerated desc
project
by
```

Meaning: Search for log clearance events in the last 7 days.

This is extremely critical because Event ID 1102 often indicates Defense Evasion (an attacker covering their tracks).

SEARCHING FOR USER CREATED

```
SecurityEvent
|      TimeGenerated > ago(7d)
|      EventID == 4720
where
where      TimeGenerated, Account, Computer, Activity
project    TimeGenerated desc
by
```

Meaning: Display user accounts that were created over the last 7 days.

Useful when investigating Persistence mechanisms or unauthorized backdoor accounts.

SEARCHING FOR USER ADDED TO GROUP

```
SecurityEvent
|      TimeGenerated > ago(7d)
|      EventID in (4728, 4732)
where  TimeGenerated, Account, Computer, Activity
where  TimeGenerated desc
project
by
```

Meaning: Search for users being added to security groups.

This becomes highly critical if the group is Administrators, Domain Admins, or Remote Desktop Users.

FINDING THE MOST FREQUENT EVENT IDS

```
SecurityEvent
  TimeGenerated > ago(24h)
|
|       EventCount=count() by EventID
where
| sort
| summarize
by
```

Meaning: Display the most frequently occurring Event IDs over the last 24 hours.

Usage:

Understanding baseline data behavior.

Discovering unusual anomaly events.

Reviewing log ingestion volumes.

HOST PIVOT: INVESTIGATING A SPECIFIC DEVICE

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      Computer contains "HR-LAPTOP-12"
where  TimeGenerated, EventID, Account, Computer, Activity
where  TimeGenerated asc
project
by
```

Meaning: Show all security activity for a specific device over the last 24 hours.

This technique is known as a Host Pivot.

USER PIVOT: INVESTIGATING A SPECIFIC USER

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      Account contains "sara"
where  TimeGenerated, EventID, Account, Computer, Activity
where  TimeGenerated asc
project
by
```

Meaning: Show all security activity for a specific user.

This technique is known as a User Pivot.

IP PIVOT: INVESTIGATING A SPECIFIC IP

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      IPAddress == "185.20.40.9"
where  TimeGenerated, EventID, Account, Computer, IPAddress, Activity
where  TimeGenerated asc
project
by
```

Meaning: Show all events associated with a specific IP address.

This technique is known as an IP Pivot.

THE ISSUE OF VARYING FIELD NAMES

If your query fails because a field is not recognized, do NOT assume the data doesn't exist.

Do this instead:

```
SecurityEvent  
| 10  
take
```

Then manually inspect the column headers. You may need to adapt your queries to match environment-specific schema names like:

IpAddress vs SourceIP

Account vs UserPrincipalName

Computer vs DeviceName

| COMMON MISTAKES IN KQL

- ✗ Forgetting the Pipe | character.
- ✗ Using = instead of == for matching.
- ✗ Searching over too large a timeframe (kills performance).
- ✗ Not using project to clean up the output.
- ✗ Not sorting results chronologically.
- ✗ Using contains too broadly (causes noise).
- ✗ Not checking the actual field names first.
- ✗ Relying on just one query to make a decision.
- ✗ Failing to translate the KQL output into actual analysis.
- ✗ Copy-pasting queries without understanding them.

IMPORTANT RULE



"KQL is not the goal. KQL is merely a tool to answer a security

question."
The goal is NOT to say: "I wrote a query."

The goal is to say: "I found 30 failed login attempts from the same IP targeting the admin account within 10 minutes, and no successful login occurred afterward."

That is analysis.

PRACTICAL APPLICATION 1

Task: Write a query that displays the first 10 records from the SecurityEvent table.

```
SecurityEvent  
|  
10  
take
```

Analytical Question: Why do we use this query?

Answer: To explore the table, understand the schema, and see the exact formatting of the fields.

PRACTICAL APPLICATION 2

Task: Write a query that displays Failed Logins from the last 24 hours.

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4625
where
where
```

Analytical Question: What is the next logical step?

Answer: Aggregate the results by Account or IpAddress to look for patterns (e.g., using 'summarize').

PRACTICAL APPLICATION 3

Task: Write a query to find the accounts with the most failed logins in the last 24 hours.

```
SecurityEvent
    TimeGenerated > ago(24h)
|
| EventID == 4625
where FailedCount=count() by Account
where FailedCount desc
| sort
| summarize
by
```

Analysis: The top account on this list might be targeted by a threat actor, or it could be a service account running with an expired/incorrect password.

PRACTICAL APPLICATION 4

Task: Write a query that shows both Failed and Successful logins for user finance.manager.

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      Account contains "finance.manager"
where  EventID in (4624, 4625)
where  TimeGenerated, EventID, Account, Computer, IPAddress, Activity
where  TimeGenerated asc
project
by
```

Goal: To chronologically determine if a successful login occurred immediately following the failed attempts.

PRACTICAL APPLICATION 5

Task: Write a query to search for encoded PowerShell executions in the last 24 hours.

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      EventID == 4688
where  CommandLine contains "-enc" or CommandLine contains "EncodedCommand"
where  TimeGenerated, Account, Computer, CommandLine
where  TimeGenerated desc
project
by
```

Analysis: If results appear, you must immediately investigate the Parent Process and any associated network connections.

PRACTICAL APPLICATION 6

Task: Write a query to look for "Audit Log Cleared" events in the last 7 days.

```
SecurityEvent
|      TimeGenerated > ago(7d)
|      EventID == 1102
where  TimeGenerated, Account, Computer, Activity
where  TimeGenerated desc
project
by
```

Analysis: Event ID 1102 is a highly critical indicator and heavily implies an attacker is actively attempting Defense Evasion.

FULL TRAINING SCENARIO

Alert: Multiple Failed Logins Followed by Successful Login

User: hr.manager | **Source IP:** 185.20.40.9

Time range: Last 24 hours

Tasks:

Search for Failed Logins for this user.

Search for Successful Logins for this user.

Build a Timeline.

Display only the essential columns.

Determine if a success occurred after the failure.

Write an Initial Analysis.

SCENARIO QUERY EXECUTION

```
SecurityEvent
|      TimeGenerated > ago(24h)
|      Account contains "hr.manager"
where EventID in (4624, 4625, 4672)
where TimeGenerated, EventID, Account, Computer, IPAddress, Activity
where TimeGenerated asc
project
by
```

Expected Analysis:

If you see many 4625 events, followed suddenly by a 4624, and then a 4672 (Special Privileges), this heavily elevates the likelihood of Account Compromise. It requires immediate escalation if the Source IP is external or the account is sensitive.

INITIAL ANALYSIS FOR THE SCENARIO

Based on the KQL timeline results, here is how an analyst writes the summary:

"The query shows multiple failed login attempts for user hr.manager from source IP 185.20.40.9, followed by a successful login within the same time window. A special privileges event was also observed after the successful login. This pattern may indicate possible account compromise, especially if the source IP is external or unusual for the user. Further investigation and escalation are recommended."

TRAINING SCENARIO 2

Alert: Suspicious PowerShell Activity

Host: HR-LAPTOP-07

Time range: Last 24 hours

Tasks:

Search for PowerShell executions.

Search specifically for -enc or EncodedCommand.

Display the User, Host, and CommandLine.

List what evidence is still missing to complete the investigation.

SCENARIO 2 QUERY EXECUTION

Broad Query:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where Computer contains "HR-LAPTOP-07"
| where EventID == 4688 and CommandLine contains "powershell"
| project TimeGenerated, Account, Computer, CommandLine
| sort by TimeGenerated asc
```

Targeted Query:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where Computer contains "HR-LAPTOP-07" and EventID == 4688
| where CommandLine contains "-enc" or CommandLine contains "EncodedCommand"
| project TimeGenerated, Account, Computer, CommandLine
| sort by TimeGenerated asc
```

MISSING EVIDENCE IN POWERSHELL ALERTS

After finding encoded PowerShell, we still need more context:

- Q What is the Parent Process? (Did it come from Word/Excel?)
- Q Did the user receive a suspicious email before this?
- Q Are there any associated DNS Queries?
- Q Are there outbound external connections?
- Q Do we have the original File Hash?
- Q Were any new Scheduled Tasks created?
- Q Did the EDR flag any related alerts?
- Q Have these commands run on other devices?

KQL provides the start of the investigation, not the end.

IN-CLASS ASSIGNMENT

Write KQL queries for the following requirements:

1. Display the first 20 records from SecurityEvent.
2. Display Failed Logins in the last 24 hours.
3. Display Successful Logins in the last 24 hours.
4. Summarize the accounts with the most Failed Logins.
5. Summarize the IP addresses causing the most Failed Logins.
6. Search for Event ID 1102 over the last 7 days.
7. Search for any account name containing "admin".
8. Search for a specific user's activity and sort chronologically.

IN-CLASS ASSIGNMENT ANSWERS (PART 1)

1. Display 20 records:

```
SecurityEvent  
| take 20
```

2. Failed Logins (24h):

```
SecurityEvent  
| where TimeGenerated > ago(24h)  
| where EventID == 4625
```

3. Successful Logins (24h):

```
SecurityEvent  
| where TimeGenerated > ago(24h)  
| where EventID == 4624
```

4. Most Failed Accounts:

```
SecurityEvent  
| where TimeGenerated > ago(24h)  
| where EventID == 4625  
| summarize FailedCount=count() by Account  
| sort by FailedCount desc
```

IN-CLASS ASSIGNMENT ANSWERS (PART 2)

5. Most Failed IPs:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where EventID == 4625
| summarize FailedCount=count() by IPAddress
| sort by FailedCount desc
```

6. Audit Log Cleared (7d):

```
SecurityEvent
| where TimeGenerated > ago(7d)
| where EventID == 1102
| project TimeGenerated, Account, Computer, Activity
```

7. Admin Accounts:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where Account contains "admin"
```

8. User Timeline:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where Account contains "sara"
| project TimeGenerated, EventID, Account, Computer, Activity
| sort by TimeGenerated asc
```

TRANSLATING RESULTS INTO ANALYSIS

The query output alone is not enough. You must translate it.

Example Output:

Account	FailedCount
admin	120
sara	8
khaled	3

The Analysis:

"The 'admin' account sustained 120 failed login attempts over the last 24 hours. This is an elevated volume requiring immediate review of the Source IPs. We must verify if any successful logins occurred following these failures and confirm if MFA is enabled and actively protecting this sensitive account."

COMMON MISTAKES IN INTERPRETING RESULTS

- ✗ Seeing a large number and panicking without context.
- ✗ Assuming every Failed Login is a targeted attack.
- ✗ Ignoring the Successful Login checks.
- ✗ Ignoring the Source IP's reputation or location.
- ✗ Forgetting that some accounts are automated Services.
- ✗ Not checking the exact time window.
- ✗ Not checking if this failure volume is a daily baseline.
- ✗ Failing to build a chronological Timeline.
- ✗ Forgetting to document the KQL query used in the ticket.
- ✗ Closing the case without writing an analytical summary.

THE GOLDEN RULE OF QUERYING



*"Any KQL query you write must yield one of the following:
Evidence, Disproof of Evidence, A New Question, A Decision, or A New
Investigative Step."*

If your query does not produce one of these, you are likely just searching randomly.

LECTURE SUMMARY

- ✓ The meaning and purpose of KQL.
- ✓ Why the SOC relies on KQL logic.
- ✓ Query syntax structure and the Pipe |.
- ✓ take to sample data.
- ✓ where to filter records.
- ✓ contains for partial string matches.
- ✓ summarize to aggregate data.
- ✓ count to get total volumes.
- ✓ ago for time window filtering.
- ✓ project to clean up columns.
- ✓ sort by to order results.
- ✓ in for multiple exact matches.
- ✓ Searching for Failed (4625) & Successful (4624) logins.
- ✓ Building chronological Timelines.
- ✓ Translating raw query output into actionable analysis.

REVIEW QUESTIONS

1. What does KQL stand for?
2. Why does a SOC analyst need KQL capabilities?
3. What is the function of the Pipe | symbol?
4. What is the function of take?
5. What is the function of where?
6. What is the difference between == and contains?
7. What is the function of summarize?

1. Why do we frequently use ago(24h)?
2. What is the function of project?
3. What is the function of sort by?
4. How do you specifically search for Failed Logins?
5. How do you specifically search for Successful Logins?
6. How do you build a chronological Timeline for a user?
7. Why is the raw query output alone insufficient for a ticket?

KNOWLEDGE ASSESSMENT QUIZ

1. The command take 10 means:

A) Delete 10 records | B) **Display 10 records** | C) Send 10 alerts | D) Close 10 Incidents

2. Event ID 4625 usually means:

A) Successful Logon | B) **Failed Logon** | C) Audit Log Cleared | D) User Created

3. where is used for:

A) **Filtering results** | B) Deleting the table | C) Changing a password | D) Opening a file

4. summarize count() by Account means:

A) **Calculating the number of events per Account** | B) Deleting the Account | C) Renaming the Account | D) Displaying only the last Account

5. ago(24h) means:

A) **The last 24 hours** | B) After 24 hours | C) Delete 24 records | D) 24 users

6. A good query execution starts with:

A) **A clear security question** | B) Random copying | C) Deleting results | D) Closing the alert

LECTURE HOMEWORK

Write KQL queries for the following scenario:

Alert Name: Possible Brute Force Success

User: finance.manager | **Source IP:** 185.20.40.9

Time Range: Last 24 hours

Query 1: Display Failed Logins for the user.

Query 2: Display Successful Logins for the user.

Query 3: Display 4624, 4625, and 4672 events for the user in a single Timeline.

Query 4: Calculate the total number of Failed Logins grouped by Source IP.

Query 5: Search for Event ID 1102 over the last 7 days.

Task 6: Write an Initial Analysis (5 to 7 lines) based on the expected outcome of the timeline.

| HOMEWORK ANSWER TEMPLATE

KQL Investigation Template:

Alert Name:

Question 1:

KQL Query:

Expected Result:

Analysis:

Question 2:

KQL Query:

Expected Result:

Analysis:

Timeline Query:

Key Findings:

Severity:

Confidence:

Verdict:

Recommended Actions:

Initial Analysis:

HOMework BRIEF ANSWER KEY

Timeline Query:

```
SecurityEvent
| where TimeGenerated > ago(24h)
| where Account contains "finance.manager"
| where EventID in (4624, 4625, 4672)
| project TimeGenerated, EventID, Account, Computer, IPAddress, Activity
| sort by TimeGenerated asc
```

Initial Analysis:

The KQL timeline query should be used to determine whether the user finance.manager experienced multiple failed login attempts followed by a successful login and special privileges assignment. If the same external source IP appears across failed and successful login events, this may indicate possible brute force success or account compromise. Event ID 4672 after the successful login increases concern because it suggests privileged activity. Additional investigation should verify MFA status, user confirmation, source IP reputation, and post-login activity. If confirmed, escalation to Tier 2 is required.

COMING UP NEXT...



Lecture 11: Incident Response Basics

Understanding how a SOC reacts to confirmed threats.

Preparation | Identification | Containment | Eradication | Recovery | Lessons Learned
What L1 Analysts *MUST* do and what they *MUST NOT* do.

INSTRUCTOR GUIDE: SUGGESTED TIME ALLOCATION

Time	Activity Focus
15 Minutes	Review SIEM concepts from the previous lecture
20 Minutes	Explain KQL concepts and why SOC analysts rely on it
25 Minutes	Query structure basics and the use of take / where
25 Minutes	Hunting specifically for Event IDs 4624 and 4625
25 Minutes	Using contains for strings and ago for timeframes
30 Minutes	Data manipulation using summarize, count, and sort
25 Minutes	Refining output with project, in, and Timeline building
30 Minutes	Practical hands-on querying applications
20 Minutes	Full Scenario Walkthrough
25 Minutes	Short Quiz, Summary, and Homework Assignment

| INSTRUCTOR GUIDE: FOCUS POINTS

- 🎯 Do not turn this lecture into a pure programming class. Keep it focused on threat hunting.
- 🎯 Tie every KQL query directly to a specific security question.
- 🎯 Do not jump into complex queries (like join or let) too early.
- 🎯 Reiterate constantly that field names (like IpAddress vs SourceIP) vary across environments.
- 🎯 Train the students heavily on reading and interpreting the output results.
- 🎯 Do not accept a query from a student without an accompanying analytical summary.
- 🎯 Focus intensely on Event IDs 4624, 4625, 4672, 4688, and 1102.
- 🎯 Ensure every student writes an Initial Analysis block after running their query.

INSTRUCTOR GUIDE: EXPECTED STUDENT MISTAKES

Mistake 1: Blind Copy-Pasting

The student copies a query without understanding it.

Correction: Ask them directly: "What security question does this query answer?"

Mistake 2: Forgetting the Time Range

Querying massive datasets without ago() limits.

Correction: Explain that without TimeGenerated, the search spans too much data and crashes performance.

Mistake 3: Ignorance of Field Names

Getting errors because they assume field names are static.

Correction: Train them to always start with take 10 to inspect the column headers.

Mistake 4: Missing the Big Picture

Seeing many Failed Logins and immediately declaring a breach.

Correction: Remind them to search for Successful Logins, check Source IPs, verify MFA status, and monitor subsequent activity.

CLOSING MESSAGE TO STUDENTS



"KQL is not just a list of commands you memorize. KQL is your voice; it is how you interrogate the logs and ask security questions."

Whose login failed? Did they succeed afterward? From which IP? On which device? What happened post-login? Were special privileges granted? Was PowerShell executed? Were the audit logs cleared?

Master the questions, and the queries will follow naturally.